

Parallel Ensemble Inference Engine

Hybrid MPI + OpenMP Parallelization of a
Soft-Voting Classifier

HPC Project Report, University of Messina

Andrea Fasci

2025-2026

<https://github.com/andre1012345/HPC-inference>

Abstract

This report presents the design and experimental evaluation of a hybrid MPI + OpenMP parallel inference engine for a three-model soft-voting ensemble classifier (K-Nearest Neighbours, Gaussian Naive Bayes, and Logistic Regression). The ensemble is trained offline on a fully synthetic binary classification dataset (70,000 samples, 20 features) and achieves 88.44% accuracy. At inference time, the test set is distributed across MPI ranks via `MPI_Scatter`; each rank parallelizes its local work with OpenMP; results are gathered with non-blocking `MPI_Isend/MPI_Irecv` and accuracy is validated with `MPI_Reduce`. Experiments on an Intel Core i5-8350U (4 physical cores, WSL2) show a best speedup of $3.27\times$ at $p = 4$ ranks and $t = 2$ threads per rank. Scaling is ultimately memory-bound: each MPI rank independently holds the full 50,000 sample training set (~ 8 MB), exceeding the 6 MB L3 cache and forcing concurrent DRAM accesses that erode efficiency at higher parallelism levels.

Contents

1	Introduction	3
1.1	Context	3
1.2	The Ensemble	3
1.3	Why Parallelize?	3
1.4	Proposed Solution	4
1.5	The Hybrid Parallelism Hierarchy	4
2	Problem Rationale	5
2.1	Problem Statement	5
2.2	Dataset	5
2.3	Sequential Solution	5
2.4	Profiling	6
2.5	Existing Parallel Approaches	6
3	Decomposition and Partitioning	7
3.1	Partitioning	7

3.2	Task Dependency Graph (TDG)	7
3.3	Task Interaction Graph (TIG)	8
4	Communication	9
4.1	MPI_Scatter for the Input Distribution	9
4.2	MPI_Isend / MPI_Irecv + MPI_Waitall for the Result Gathering	9
4.3	MPI_Reduce for the Accuracy Check	10
4.4	Summary	10
5	Agglomeration	10
5.1	Granularity	10
5.2	KNN Buffer Strategy	10
5.3	Training Data Locality	11
6	Mapping	11
6.1	Hardware Platform	11
6.2	Configurations Tested	11
7	Performance Evaluation	12
7.1	Setup	12
7.2	Results	13
7.3	Analysis	14
8	Conclusion	15
	References	16

1 Introduction

1.1 Context

Batch inference : classifying a large set of unlabelled samples against a pre-trained model, is a routine workload in production machine-learning pipelines. When one of the models is a brute-force K-Nearest Neighbours classifier, the cost is $O(N_{\text{train}} \times F)$ per query: every test sample must be compared against all training points across all features. At the scale used here : 20,000 queries against 50,000 training points with 20 features , this amounts to roughly 2×10^{10} floating-point operations, making sequential execution too slow for any interactive use case (baseline wall time: 28.3 s on a four-core laptop).

The problem is a target for data-parallel acceleration: each test sample is fully independent, so the work can be distributed across workers without any inter-query communication during inference. The question addressed in this project is *how* to structure that parallelism across the two programming models available on a multi-core machine: MPI for inter-process distribution and OpenMP for intra-process threading.

1.2 The Ensemble

Three classifiers are combined into a soft-voting ensemble:

- **KNN (K-Nearest Neighbours):** stores all 50,000 training points and computes brute-force distances at query time. This is the *dominant cost* : about 87% of total runtime. Classification accuracy of the algorithm alone: 93.25%.
- **Naive Bayes (NB):** uses precomputed class means and variances. Each prediction is a few arithmetic operations (essentially free). Classification accuracy of the algorithm alone: 79.64%.
- **Logistic Regression (LR):** a dot product plus a sigmoid. Also negligible in cost. Classification accuracy of the algorithm alone: 80.50%.

The three predicted probabilities are averaged; if the average is ≥ 0.5 the sample is classified as class 1. Combining the three models raises accuracy to **88.44%**, above NB and LR while staying close to KNN, which justifies the ensemble.

1.3 Why Parallelize?

KNN dominates the computational budget. Here $N_{\text{train}} = 50,000$ is the number of training points, $F = 20$ the number of features, and $C = 2$ the number of classes. Each test query requires a full scan of the training set , one squared-distance computation per training point , followed by a `partial_sort` to extract the k nearest neighbours. The per-query cost is $O(N_{\text{train}} \times F)$, giving a total of:

$$20,000 \times 50,000 \times 20 \approx 2 \times 10^{10} \text{ multiply-accumulate operations}$$

LR and NB contribute $O(F)$ and $O(C \times F)$ per query respectively , four to five orders of magnitude less, and are effectively free by comparison.

Critically, test queries are mutually independent: the prediction for sample i depends only on `X_train` and the model parameters, not on any other test sample. This structure means the workload can be partitioned across processes and threads with zero inter-query synchronization, making parallelization both natural and efficient.

1.4 Proposed Solution

The hybrid binary `inference_dist_knn.cpp` partitions the inference workload across two levels of parallelism:

1. **MPI_Scatter** distributes the test set evenly across p ranks; each rank receives N_{test}/p samples and holds a full copy of `X_train`, preserving exact KNN accuracy.
2. **OpenMP** parallelises the per-sample inference loop inside each rank with a single `parallel for` (schedule static), one fork/join per timed run rather than per sample.
3. **MPI_Isend/MPI_Irecv + MPI_Waitall** gather local predictions at rank 0 non-blockingly, overlapping communication with the implicit OpenMP barrier.
4. **MPI_Reduce** aggregates per-rank correct-prediction counts for a single global accuracy figure.

The two-level design has a concrete memory implication: each MPI rank holds one independent copy of `X_train`. On the test machine ($p \leq 4$), this means at most four 8 MB copies in DRAM simultaneously, a deliberate trade-off between memory footprint and the complexity of distributing the training set across ranks, which would require approximate rather than exact KNN.

1.5 The Hybrid Parallelism Hierarchy

The two programming models are not interchangeable: they operate at different granularities and carry fundamentally different memory semantics.

MPI parallelism is *coarse-grained*. Each rank is an independent OS process with its own address space; inter-rank communication requires explicit messaging. This makes MPI robust against false sharing and cache-coherence traffic, but each rank must hold its own copy of `X_train`. Scaling MPI ranks alone multiplies memory pressure proportionally.

OpenMP parallelism is *fine-grained*. Threads within a rank share the same address space, so `X_train` and the model parameters are loaded once and read concurrently without copying. The cost is potential contention on shared data structures — avoided here because each thread writes to a disjoint index of `local_preds[i]` and uses pre-allocated private buffers (`dist_bufs[tid]`, `idx_bufs[tid]`).

The hybrid design exploits both properties simultaneously: MPI distributes the test set across p processes (reducing per-rank work), while OpenMP extracts parallelism within each process without multiplying the memory footprint. The total worker count is $p \times t$; Figure 1 illustrates this for $p = 2$, $t = 2$.



Figure 1: The two-level hybrid hierarchy for $p = 2$, $t = 2$. MPI partitions the 20,000 test samples into rank-level slices (coarse-grained, distributed memory); OpenMP subdivides each slice among threads (fine-grained, shared memory).

2 Problem Rationale

2.1 Problem Statement

Input: a pre-trained ensemble (KNN, NB, LR) and a test set of 20,000 samples with 20 features each.

Output: one binary prediction per sample, matching the accuracy of the sequential implementation.

Goal: minimize wall-clock time on a multi-core machine.

2.2 Dataset

The dataset is **fully synthetic**, generated with scikit-learn:

```

1 X, y = make_classification(
2     n_samples=70_000, n_features=20,
3     n_informative=10, n_redundant=2,
4     n_classes=2, class_sep=1.0, random_state=42
5 )

```

The 70,000 samples are split 50,000 / 20,000 (train / test) with stratified sampling. Features are standardised with a `StandardScaler` fitted on training data only. Scaler parameters are exported to `model_params/` so the C++ engine operates on pre-scaled data.

A synthetic dataset was chosen for full reproducibility and to avoid any licensing or data-provenance issues.

2.3 Sequential Solution

The sequential baseline processes one test sample at a time:

1. `knn_predict_proba` : iterates over all 50,000 training points, computes squared Euclidean distances, runs `partial_sort` to find the $k = 5$ nearest neighbours.
2. `nb_predict_proba` : log-posterior with log-sum-exp normalisation.

3. `lr_predict_proba` : $\sigma(\mathbf{w} \cdot \mathbf{x} + b)$.

The KNN distance and index scratch buffers (50,000 elements each) are allocated *once* before the run loop and reused across all samples.

2.4 Profiling

The sequential binary was compiled with `-O0 -pg` and profiled with `gprof` to find the hotspot. Table 1 shows the result.

Table 1: `gprof` flat profile top entries. KNN alone accounts for 87.15% of runtime.

% time	Function	Calls
87.15	<code>knn_predict_proba</code>	100,000
3.39	<code>comparison lambda (partial_sort)</code>	720,174,764
2.56	<code>std::iota</code>	100,000
1.91	<code>_Iter_comp_iter::operator()</code>	713,250,144
1.78	<code>vector::data()</code>	705,032,704
1.71	<code>std::__heap_select</code>	100,000
0.02	<code>nb_predict_proba</code>	100,000
0.00	<code>lr_predict_proba</code>	100,000

NB and LR together account for less than 0.03% of runtime. All parallelization effort is therefore correctly directed at the outer loop over test samples, where `knn_predict_proba` is called.

The binary was compiled with `-O0` to prevent the compiler from inlining or collapsing call chains, which would hide individual functions from the sampler. The absolute times are consequently $\sim 5\times$ slower than the optimized baseline and carry no performance significance; the profile is used solely to confirm the hotspot. All benchmarks in Section 7 use `-O2`.

2.5 Existing Parallel Approaches

Three strategies are commonly used to parallelise brute-force KNN inference; the choice between them has direct consequences for communication overhead and accuracy.

Test-set decomposition assigns each worker a disjoint slice of the test set while every worker retains the full training set. No inter-worker communication occurs during the inference loop; a single scatter before and a single gather after are sufficient. Giacomazzi et al. [1] demonstrate that a three-level hierarchy of threads, MPI processes, and Grid nodes built on this decomposition achieves speedups exceeding $15\times$ for large-scale epidemiological classification workloads. This is the strategy adopted in this project.

Training-set decomposition partitions `X_train` across workers; each computes partial distances, and a reduce step merges the global k nearest neighbours before classification. This eliminates the per-rank copy of training data but introduces communication on the critical path of every query. Ignatij [2] implements both strategies with MPI and reports that test-set decomposition consistently outperforms training-set decomposition, motivating the choice made here.

Approximate KNN replaces exact search with index structures — trees, graphs, or quantization, reducing per-query cost from $O(N_{\text{train}})$ to $O(\log N_{\text{train}})$ at the cost of recall. Johnson et al. [3] show that FAISS scales to billion-scale datasets with controllable accuracy loss. For this project, however, exact reproducibility of the sequential accuracy (88.44% across all configurations) was a hard requirement, which excludes approximate methods by definition.

3 Decomposition and Partitioning

The design follows Foster’s PCAM methodology (Partition, Communicate, Agglomerate, Map).

3.1 Partitioning

Domain decomposition is applied over the test set at two levels, mirroring the hybrid hierarchy described in Section 1.5.

MPI level (coarse). The 20,000 samples are split into p equal chunks, one per MPI rank. Each chunk is independent: the inference for sample i depends only on $\mathbf{x}_i$ and the shared read-only model parameters.

OpenMP level (fine). Within each rank, the N_{test}/p local samples are further subdivided among t threads by the `schedule(static)` clause, which assigns contiguous blocks, optimal for the sequential memory layout of `X_test_local`.

This gives up to 20,000 tasks, more than four orders of magnitude larger than the maximum process count tested ($p = 4$), which satisfies Foster’s requirement.

Functional decomposition (one model per rank) was considered but rejected: since KNN completely dominates, splitting the models would leave non-KNN ranks idle almost all the time.

3.2 Task Dependency Graph (TDG)

The TDG models the computational structure of the problem in terms of tasks and their logical dependencies, independently of any implementation technology. MPI ranks and OpenMP threads are not visible here; they appear later in the Mapping phase.

Algorithm-level structure. The inference problem decomposes into N independent *sample groups*. Each group i contains three mutually independent compute tasks (KNN_i , NB_i , LR_i) that all read the same $\mathbf{x}_i$ and feed a per-sample vote task (Vote_i). All N vote results feed a single Aggregate task which computes global accuracy.

Two structural properties are immediately visible from the graph:

1. **No inter-sample dependencies.** Sample group i and group j share no data flow. The N groups can execute in any order or simultaneously. This is the property that the domain decomposition exploits directly.
2. **Single global synchronisation point.** Aggregate is the only node that depends on the entire computation. It cannot begin until all N Vote tasks complete, making it the irreducible serial fraction. This corresponds to the $\approx 13\%$ floor estimated by Amdahl’s Law.

Critical path. Since all sample groups have identical cost (each KNN task searches the same 50,000 training points), every root-to-leaf path in the TDG has equal length. The critical path is therefore:

$$\text{KNN}_i \longrightarrow \text{Vote}_i \longrightarrow \text{Aggregate}$$

for any i . Its length sets the minimum achievable parallel execution time regardless of how many processors are available.

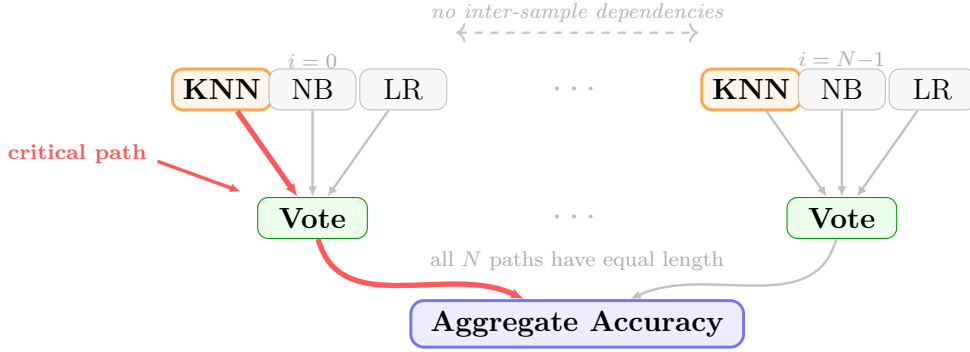


Figure 2: Algorithm-level Task Dependency Graph (two of the N sample groups shown).

Intra-sample structure and why it is not parallelised. Within a single sample group, KNN, NB, and LR are mutually independent and could in principle run concurrently. However, KNN accounts for $\approx 87\%$ of per-sample cost (Section 2.4), so it defines the intra-sample critical path by itself. Running NB and LR in parallel with KNN would shorten per-sample time by at most 13%, while forcing a thread fork/join on every one of the $N = 20,000$ samples per run. The overhead dominates the gain. The correct level at which to apply parallelism is *across* sample groups, not within them.

3.3 Task Interaction Graph (TIG)

The TIG shows who communicates with whom at runtime. In this implementation the answer is simple: only Rank 0 exchanges data with the other ranks. Before inference it sends each worker its test slice; after inference it collects the predictions. Workers never talk to each other directly.

This produces a **star topology**: all $p - 1$ communication edges connect to Rank 0, and the total message count is $O(p)$. Doubling the number of ranks doubles the number of messages, a linear cost.

The contrast with training-set decomposition is instructive. If each rank held only a shard of $\mathbf{X}_{\text{train}}$, it would need to compare its partial distances with every other rank to find the global k nearest neighbours. Every rank would need a direct channel to every other rank, producing a fully-connected graph with $O(p^2)$ edges. At $p = 8$ that is already 28 connections instead of 7. Test-set decomposition avoids this entirely by keeping each rank's inference self-contained.

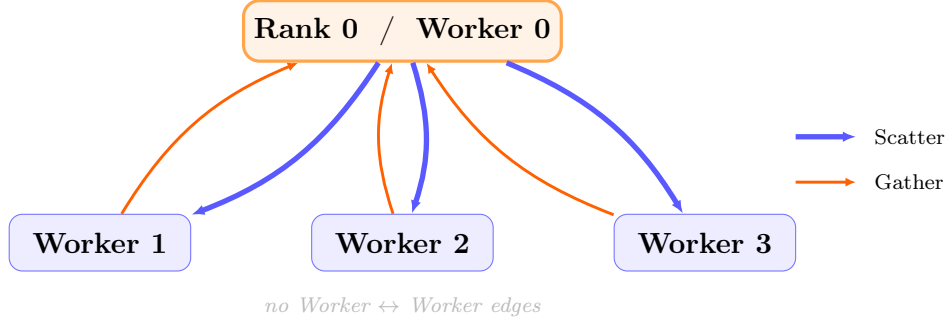


Figure 3: Task Interaction Graph for $p = 4$. The star topology has $O(p)$ edges total.

4 Communication

Three MPI patterns are used, each for a different purpose.

4.1 MPI_Scatter for the Input Distribution

Before the timed loop, rank 0 splits and sends the test data:

```

1 MPI_Scatter(X_test_all.data(),
2             local_n_test * N_FEATURES, MPI_DOUBLE,
3             X_test_local.data(),
4             local_n_test * N_FEATURES, MPI_DOUBLE,
5             0, MPI_COMM_WORLD);

```

`MPI_Scatter` is a one-to-all collective: rank 0 splits a contiguous buffer into equal chunks and sends one to each rank. The test set is trimmed to a multiple of p before scattering. This call is *outside* the timing loop.

4.2 MPI_Isend / MPI_Irecv + MPI_Waitall for the Result Gathering

After inference, predictions are gathered at rank 0 using non-blocking point-to-point operations:

```

1 if (rank == 0) {
2     std::copy(local_preds.begin(), local_preds.end(),
3               global_preds.begin());
4     std::vector<MPI_Request> reqs(size - 1);
5     for (int r = 1; r < size; ++r)
6         MPI_Irecv(global_preds.data() + r * local_n_test,
7                   local_n_test, MPI_INT,
8                   r, 0, MPI_COMM_WORLD, &reqs[r - 1]);
9     MPI_Waitall((int)reqs.size(), reqs.data(),
10                MPI_STATUSES_IGNORE);
11 } else {
12     MPI_Request req;
13     MPI_Isend(local_preds.data(), local_n_test,
14               MPI_INT, 0, 0, MPI_COMM_WORLD, &req);

```

```

14 MPI_Wait(&req, MPI_STATUS_IGNORE);
15 }

```

Rank 0 posts all receives at once (potentially overlapping multiple transfers), then `MPI_Waitall` blocks until all are done.

4.3 MPI_Reduce for the Accuracy Check

Each rank counts how many of its local predictions are correct. A single `MPI_Reduce` sums the counts at rank 0:

```

1 MPI_Reduce(&local_correct, &global_correct, 1,
2           MPI_LONG, MPI_SUM, 0, MPI_COMM_WORLD);

```

This is outside the timing loop and communicates one integer per rank, instead of sending individual labels again.

4.4 Summary

Table 2: The three MPI patterns and their roles.

Pattern	Type	Purpose
<code>MPI_Scatter</code>	collective, blocking	distribute test data
<code>MPI_Isend/Irecv + Waitall</code>	point-to-point, non-blocking	gather predictions at rank 0
<code>MPI_Reduce</code>	collective, blocking	sum correct counts for accuracy

5 Agglomeration

5.1 Granularity

At the finest level, each of the 20,000 test samples is an independent task. One MPI rank per sample is impractical, the spawn and communication overhead would dominate. Tasks are therefore agglomerated in two stages:

1. **MPI level.** All N_{test}/p samples for a rank are processed as one block. MPI message count is $O(p)$, independent of N_{test} .
2. **OpenMP level.** Within each rank, the loop over local samples is parallelised with a single `#pragma omp parallel for`. This gives exactly *one* fork/join per run — not one per sample.

5.2 KNN Buffer Strategy

Each OpenMP thread gets its own distance buffer and index buffer (both of length 50,000), allocated once before the timing loop and indexed by `omp_get_thread_num()`. This eliminates up to 100,000 heap allocations per run and avoids false sharing between threads.

5.3 Training Data Locality

Every rank loads `X_train` independently from disk rather than having rank 0 broadcast it. Each rank’s copy sits in its own memory region, so all p ranks stream through training data in parallel without contending for the same cache lines.

6 Mapping

6.1 Hardware Platform

The experiments run on an Intel Core i5-8350U with 4 physical cores and 8 logical threads (Hyper-Threading), under Ubuntu 22.04 / WSL2 on Windows 11.

Figure 4 shows the memory hierarchy. The most important detail is that the KNN working set ($50,000 \times 20 \times 8 = 8$ MB) **exceeds the 6 MB L3 cache**. Every KNN call therefore reads from DRAM, making the kernel memory-bandwidth-bound. This is the main reason speedup does not scale linearly beyond 4 physical cores.

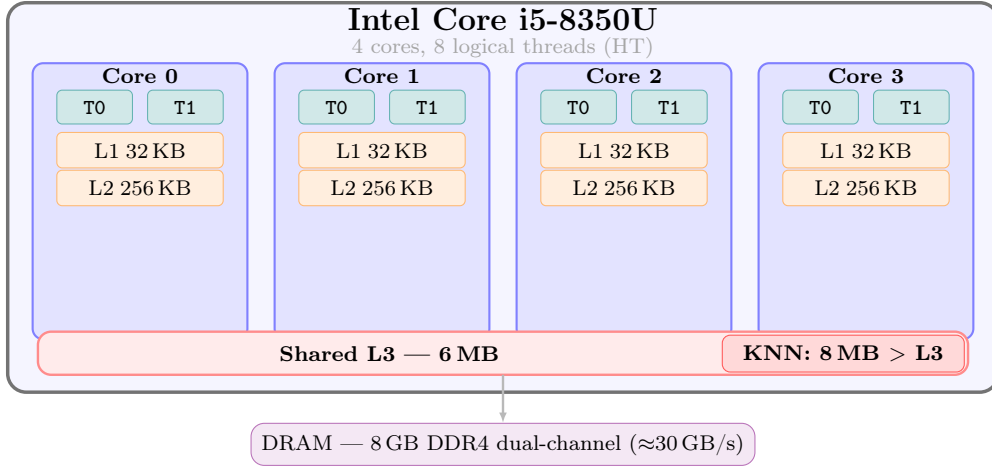


Figure 4: Memory hierarchy of the test platform. L1/L2 caches are private per core; the 6 MB L3 is shared. The KNN training data (8 MB) exceeds the L3, forcing all threads to stream from DRAM and saturating the shared memory bus as core count grows.

6.2 Configurations Tested

No explicit CPU affinity is set; the OS scheduler assigns cores. The configurations tested are listed below, where p = MPI ranks and t = OpenMP threads per rank. All configurations use the same hybrid binary `inference_dist_knn`; $t = 1$ simply means OpenMP parallelism is inactive within each rank.

Table 3: Tested configurations. Each MPI rank maps to one physical core; $t = 2$ adds the Hyper-Threading sibling of that core.

p	t	$p \cdot t$	Notes
1	1	1	sequential baseline
1	2	2	OpenMP-level parallelism only
2	1	2	MPI-level parallelism only, 2 physical cores
2	2	4	hybrid, 2 cores + HT siblings
3	1	3	MPI-level parallelism only, 3 physical cores
4	1	4	MPI-level parallelism only, all 4 physical cores
3	2	6	hybrid, 3 cores + HT siblings
4	2	8	hybrid, all 4 cores + HT siblings

Figure 5 shows how the two $p = 4$ configurations map onto the physical hardware.

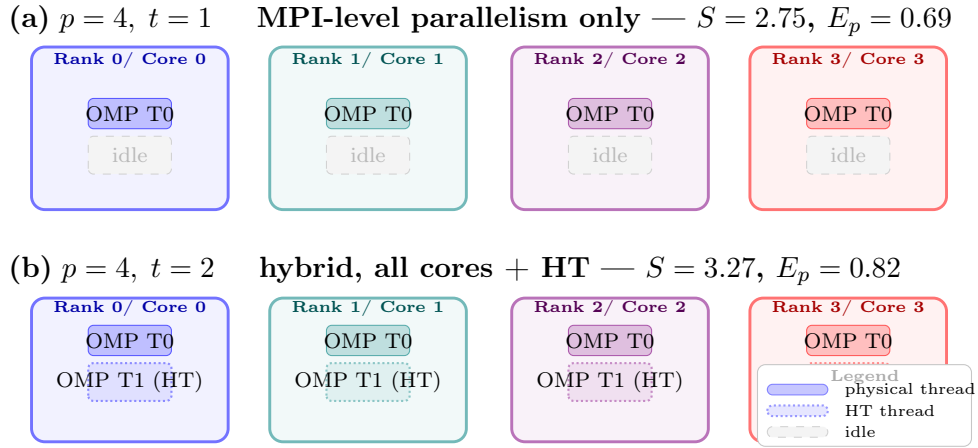


Figure 5: The two $p = 4$ configurations compared on the same hardware. (a) $t = 1$: one OpenMP thread per rank, HT slots idle, parallelism operates at MPI level only. (b) $t = 2$: the second OpenMP thread in each rank occupies the HT sibling, sharing execution units with T0. Adding HT raises speedup from $S = 2.75$ to $S = 3.27$ (+19%) while keeping the same 4 physical cores. Core placement is OS-managed.

7 Performance Evaluation

7.1 Setup

- **Machine:** Intel Core i5-8350U, 4 physical cores, 8 logical threads, 6 MB L3, 15 W TDP, Ubuntu 22.04 / WSL2.
- **Compiler:** `mpic++`, optimisation `-O2` for all binaries (including the sequential baseline).
- **Measurement:** 3 runs per configuration; median time reported. Each run starts with `MPI_Barrier` to ensure all ranks begin timing simultaneously before `MPI_Wtime()` is called.
- **Baseline:** $T_1 = 28.3243$ s (sequential, $p = 1, t = 1$).

- **Metrics:** speedup $S_p = T_1/T_p$; efficiency $E_p = S_p/p$, where p is the number of *physical cores* (MPI ranks). HT threads are not counted as independent processors in the efficiency denominator: SMT siblings share fetch/decode units and are not equivalent to additional physical cores.
- **Note on WSL2:** experiments run under a virtualisation layer. Absolute timings may be affected by scheduling overhead, but relative scaling trends remain meaningful.

7.2 Results

Table 4: Benchmark results. $T_1 = 28.3243$ s. Efficiency $E_p = S_p/p$ uses physical cores p as the denominator. Values $E_p > 1$ for $t = 2$ configurations reflect the additional throughput contributed by the HT sibling threads, which are not counted in p .

Ranks (p)	Threads (t)	$p \cdot t$	Time (s)	Speedup S_p	Eff. $E_p = S_p/p$
1	1	1	28.3243	1.00	1.00
1	2	2	15.0657	1.88	1.88
2	1	2	14.9054	1.90	0.95
2	2	4	9.5656	2.96	1.48
3	1	3	12.6595	2.24	0.75
4	1	4	10.3024	2.75	0.69
3	2	6	8.9910	3.15	1.05
4	2	8	8.6497	3.27	0.82

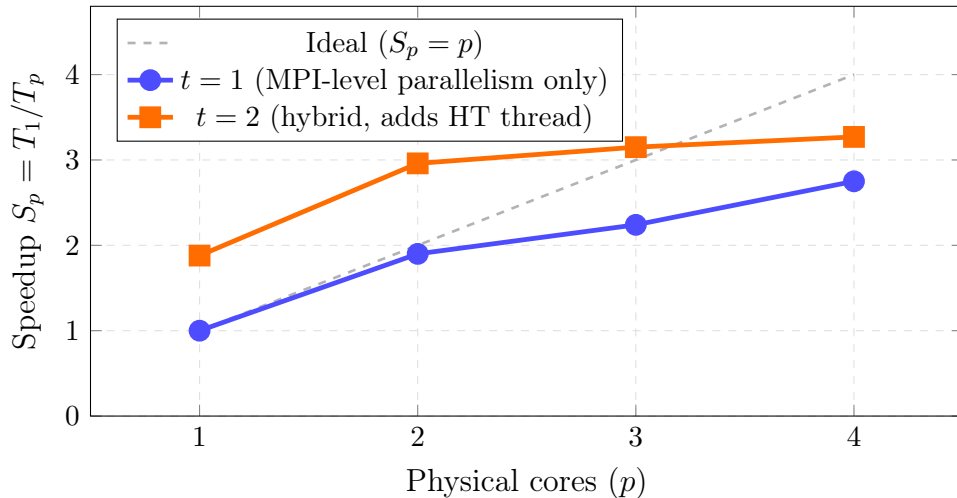


Figure 6: Speedup vs. physical cores p . The $t = 1$ series uses only MPI-level parallelism; the $t = 2$ series adds one OpenMP HT thread per rank. Both are plotted against the same hardware resource (physical cores), making them directly comparable. The hybrid series lies consistently above $t = 1$ at every p : the HT sibling thread provides additional throughput on the same core. The gap narrows at $p = 4$ because shared DRAM bandwidth becomes the bottleneck. Exact values are in Table 4.

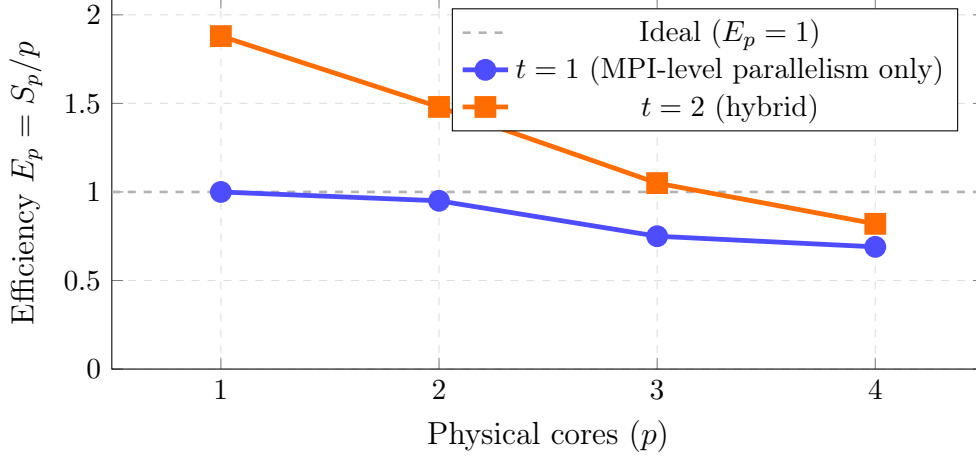


Figure 7: Efficiency $E_p = S_p/p$ vs. physical cores. The $t = 1$ series stays below the ideal line and declines with p . The $t = 2$ series starts above 1 at low p : HT threads provide throughput that exceeds the physical-core baseline, so $E_p > 1$ is physically meaningful rather than an artefact. Both series converge as p increases and shared DRAM bandwidth becomes the limiting factor.

7.3 Analysis

Physical-core baseline ($t = 1$). At $p = 2$ the $t = 1$ configuration achieves $S = 1.90$, $E_p = 0.95$, near-ideal utilization of two independent cores. Each rank operates in a fully independent address space with no shared heap, no OpenMP bookkeeping, and no risk of false sharing. Efficiency degrades to $E_p = 0.69$ at $p = 4$ as all four cores compete for the shared DRAM bus.

HT contribution ($t = 2$). Adding one HT thread per rank consistently raises speedup at every physical core count:

- $p = 1$: S rises from 1.00 to 1.88 (+88%). A single HT thread nearly doubles throughput because one thread can use execution units while the other stalls on DRAM loads.
- $p = 2$: S rises from 1.90 to 2.96 (+56%).
- $p = 3$: S rises from 2.24 to 3.15 (+41%).
- $p = 4$: S rises from 2.75 to 3.27 (+19%). The marginal gain shrinks because all HT siblings now compete on an already-saturated DRAM bus.

The HT gain exceeds one physical-core baseline ($E_p > 1$) at $p \leq 3$, which is physically meaningful: SMT extracts additional instruction-level parallelism from the memory-latency stalls that dominate the KNN kernel.

Arithmetic intensity and the memory wall. Each distance computation in KNN performs approximately 40 floating-point operations per training point (one subtraction and one multiply-accumulate per feature, across 20 features), while reading 160 bytes (20 double-precision values). This yields an arithmetic intensity of $40/160 = 0.25$ FLOP/byte. The DDR4 dual-channel bus delivers ≈ 29.9 GB/s, implying a peak throughput of only $29.9 \times 0.25 \approx 7.5$ GFLOP/s, far below the CPU’s compute ceiling. The kernel is therefore *bandwidth-bound by construction*: beyond 4 physical cores the DRAM bus is saturated

and additional threads contend rather than compute.

Amdahl’s Law. The serial fraction is estimated from the $t = 1$ best result, which is free from HT effects and directly tied to p physical processors. For $S_p = 2.75$ at $p = 4$:

$$f = \frac{1/S_p - 1/p}{1 - 1/p} = \frac{0.364 - 0.250}{0.750} \approx 0.15$$

The implied ceiling is $1/f \approx 6.7\times$ on infinite physical cores. The profiler reports $\approx 13\%$ truly serial code (file I/O, scatter, gather), consistent with this estimate: the gap between 0.15 and 0.13 is explained by memory-bandwidth contention, which serialises parallel threads through the shared DRAM bus and inflates the apparent serial fraction beyond what static profiling captures.

Why the hybrid advantage grows at scale. On the single-node platform tested here, the hybrid’s primary gain is Hyper-Threading: the second OpenMP thread occupies the HT sibling of each physical core, extracting instruction-level parallelism from DRAM-latency stalls. At $p = 1$ this alone yields $S = 1.88$. As p grows, however, the DRAM bus saturates and the HT margin shrinks (from $+88\%$ at $p = 1$ to $+19\%$ at $p = 4$).

The deeper benefit of the hybrid hierarchy emerges on a multi-node cluster. A flat deployment at $p \times t$ ranks would require $p \times t$ independent copies of `X_train` in memory and $p \times t - 1$ gather messages. The hybrid reduces both: p ranks share training data among t threads each, so only p copies reside in memory and only $p - 1$ messages are sent. On a 16-node cluster, $p = 16$ with $t = 8$ means 16 copies instead of 128 and a proportionally smaller gather phase, a structural advantage that the current single-node experiments cannot fully demonstrate but that the design already realises.

8 Conclusion

A hybrid MPI + OpenMP parallel inference engine was designed, implemented, and evaluated for a three-model soft-voting ensemble. The architecture follows a deliberate two-level hierarchy: MPI distributes the test set across p independent processes (coarse-grained, distributed memory), while OpenMP subdivides each local slice among t threads (fine-grained, shared memory). Total concurrency is $p \times t$.

The three MPI patterns required by the course specification are all present: `MPI_Scatter` for input distribution, non-blocking `MPI_Isend/MPI_Irecv + MPI_Waitall` for result gathering, and `MPI_Reduce` for accuracy validation.

Key results on the Intel Core i5-8350U, with speedup and efficiency measured against physical cores p :

- Best speedup: $3.27\times$ at $p = 4, t = 2$.
- Best $t = 1$ efficiency: $E_p = 0.95$ at $p = 2, t = 1$.
- HT contribution at $p = 4$: raises speedup from 2.75 to 3.27 ($+19\%$), with $E_p = 0.82$.
- Ensemble accuracy: **88.44%** preserved across all configurations.

The dominant bottleneck is arithmetic intensity (≈ 0.25 FLOP/byte), which makes the KNN kernel bandwidth-bound on the shared DRAM bus. The hybrid architecture’s real advantage appears at scale: on a multi-node cluster each MPI rank has a dedicated

memory bus, so the hybrid reduces inter-node message count while OpenMP exploits intra-node shared memory , achieving lower communication overhead and fewer redundant copies of `X_train` compared to a flat single-threaded MPI deployment at the same total core count.

References

- [1] P. Giacomazzi et al., “A Parallel Implementation of the K Nearest Neighbours Classifier in Three Levels: Threads, MPI Processes and the Grid,” *Parallel Computing*, 2006.
- [2] I. Ignatij, “Parallel implementation of kNN using MPI,” GitHub repository, 2022.
<https://github.com/ignatij/knn-mpi>
- [3] J. Johnson, M. Douze, H. Jégou, “Billion-Scale Similarity Search with GPUs,” *arXiv:1702.08734*, 2017.